UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, DECANA DE AMÉRICA)

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Tema

"Métodos de Búsqueda en Espacios de Estados"

Curso

Inteligencia Artificial

Profesor

Hugo Vega

Alumnos

Calero Falcon, Gonzalo Manuel

Huanacuni Gomez, Jean Carlos Josue

Lima, Perú

2026

Índice

1. Resumen 4
2. Introducción: ¿Por qué resolver problemas en computación es como buscar en un laberinto? 4
3. Conceptos Fundamentales: La Resolución de Problemas como Búsqueda 5
3.1. ¿Qué es un problema de búsqueda? De la situación inicial al objetivo deseado 6
3.2. Los cinco componentes formales de cualquier problema explicados fácil 6
3.3. Ejemplos clásicos y cotidianos: El 8-puzzle, las jarras de agua y la navegación en mapas 7
3.4. La abstracción en la ingeniería: Cómo simplificar la realidad sin perder lo esencial 8
4. Espacios de Estados y Criterios de Evaluación de Algoritmos 8
4.1. El grafo del mundo frente al árbol de búsqueda en la memoria 8
4.2. La frontera de búsqueda y el conjunto de explorados: Evitando caminar en círculos 9
4.3. Los cuatro jueces de un algoritmo: Completitud, Optimalidad, Tiempo y Memoria 10
4.4. El monstruo de la explosión combinatorial: Por qué la fuerza bruta colapsa 10
5. Algoritmos de Búsqueda Ciega (No Informada): Explorando sin Brújula 11
5.1. Búsqueda en Anchura (BFS): Exploración en ondas concéntricas como una gota de agua 11
5.2. Búsqueda de Costo Uniforme (UCS / Dijkstra): Buscando siempre el camino más económico 12
5.3. Búsqueda en Profundidad (DFS): El explorador audaz que llega hasta el fondo del abismo 13
5.4. Búsqueda de Profundidad Limitada (DLS) e Iterativa (IDS): La combinación perfecta 13
5.5. Búsqueda Bidireccional: Cavando el túnel desde ambos extremos 14
5.6. Tabla comparativa maestra de los métodos de búsqueda ciega 14
6. Algoritmos de Búsqueda Heurística (Informada): La Brújula Matemática 15
6.1. ¿Qué es una heurística? La intuición humana convertida en número 16
6.2. Propiedades indispensables: Heurísticas Admisibles y Consistentes (Monótonas) 16
6.3. Búsqueda Voraz Primero el Mejor (Greedy): El atajo miope que puede salir muy caro 17
6.4. El Algoritmo A* (A-Estrella): La obra maestra de Hart, Nilsson y Raphael (1968) 18
6.5. ¿Cómo inventar buenas heurísticas? La técnica de relajar las reglas del juego 19
6.6. Cuadro comparativo: Métodos Ciegos frente a Métodos Heurísticos 19
7. Caso Práctico de Ingeniería: Optimización de Rutas Logísticas en Lima 20
7.1. Descripción del problema de distribución urbana y red vial metropolitana 21
7.2. Modelado formal del problema: Nodos, distancias reales y estimaciones en línea recta 21
7.3. Ejecución paso a paso comparando BFS, Dijkstra, Greedy y A* 22
7.4. Tabla de resultados comparativos: Nodos expandidos, costo final y tiempo de

cómputo 23
8. Aplicaciones Reales en la Vida Diaria y los Servicios Públicos 24
8.1. Sistemas de Navegación GPS y Tránsito (Google Maps, Waze y ruteo de flotas) 24
8.2. Videojuegos y Robótica Móvil: Navegación de personajes y robots de almacén 24
8.3. Telecomunicaciones e Internet: Ruteo óptimo de paquetes de datos (OSPF) 25
8.4. Sector Eléctrico y Servicios Públicos en el Perú: Despacho económico de energía (COES) 25
9. La Situación Actual de la Búsqueda: De Deep Blue a AlphaGo y los Modelos de Lenguaje 25
9.1. La unión de Búsqueda y Machine Learning: Búsqueda en Árbol de Monte Carlo (MCTS) 26
9.2. Cómo razonan los nuevos modelos de IA: Búsqueda en Árbol de Pensamientos (Tree of Thoughts) 26
9.3. Eficiencia energética, límites computacionales y gobernanza tecnológica 27
10. Preguntas Clave para Entender la Búsqueda sin ser Experto 27
11. Conclusiones: 29
12. Referencias Bibliográficas 30

1. Resumen

El presente informe académico examina con profundidad conceptual, rigor metodológico y accesibilidad pedagógica los **Métodos de Búsqueda en Espacios de Estados**, pilares algorítmicos sobre los que se fundamenta la resolución automatizada de problemas en la Inteligencia Artificial. El documento se articula en torno a cuatro ejes cardinales. En primer lugar, se formaliza la formulación canónica de problemas mediante sus cinco componentes esenciales (estado inicial, acciones disponibles, modelo de transición, prueba de meta y costo de camino), desmitificando cómo una computadora modela laberintos, acertijos y redes de transporte reales mediante procesos de abstracción. En segundo lugar, se analiza la estructura de los espacios de estados y los criterios indispensables de evaluación algorítmica: completitud, optimalidad, complejidad temporal y complejidad espacial, evidenciando por qué el fenómeno de la explosión combinatorial hace inviable la búsqueda ciega por fuerza bruta en problemas reales. En tercer lugar, se examinan sistemáticamente los algoritmos de búsqueda no informada (búsqueda en anchura, costo uniforme, profundidad, profundidad limitada e iterativa), comparando sus fortalezas y consumos de memoria. En cuarto lugar, se profundiza en la búsqueda heurística o informada, analizando la función matemática de evaluación, los requisitos de admisibilidad y consistencia, y el célebre algoritmo A* (A-Estrella). El informe culmina con el desarrollo de un caso práctico de ingeniería sobre optimización de rutas logísticas de última milla en Lima Metropolitana, la revisión de aplicaciones industriales en transporte, videojuegos y redes eléctricas, y un examen sobre las fronteras de la búsqueda en la era moderna, destacando la integración de la búsqueda en árbol de Monte Carlo (MCTS) en AlphaGo y el razonamiento en árbol de pensamientos (*Tree of Thoughts*) en los modelos fundacionales contemporáneos.

Palabras Clave: Búsqueda en Espacios de Estados, Algoritmos Ciegos, Algoritmo A*, Función Heurística, Admisibilidad, Búsqueda en Anchura, Explosión Combinatorial, Optimización de Rutas.

2. Introducción: ¿Por qué resolver problemas en computación es como buscar en un laberinto?

Casi todos los desafíos complejos a los que nos enfrentamos los seres humanos en nuestra vida diaria son, en el fondo, problemas de búsqueda. Imaginemos que nos encontramos en medio de un laberinto gigante de setos verdes en un parque de diversiones. Estamos parados en la entrada (nuestro punto de partida) y queremos llegar a la glorieta central (nuestro objetivo). En cada esquina tenemos que tomar una decisión: doblar a la izquierda, seguir de frente o doblar a la derecha. Si doblamos al azar sin ningún orden, es muy probable que demos vueltas en círculos durante horas o terminemos en pasajes sin salida. Para salir airosos, necesitamos una estrategia: recordar qué pasillos ya recorrimos, evitar repetir caminos y, si es posible, subirnos a una banca para mirar por encima de los arbustos y ver en qué dirección asoma el techo de la glorieta.

En las ciencias de la computación y en la Inteligencia Artificial ocurre exactamente lo mismo. Cuando le pedimos a una computadora que resuelva un problema —ya sea encontrar la ruta más rápida para una ambulancia entre San Juan de Lurigancho y el Hospital Rebagliati en Lima, organizar el horario de despegue de cientos de aviones en un aeropuerto internacional, resolver una partida de ajedrez o descifrar el cubo de Rubik—, la máquina no "adivina" la solución de forma mágica. Lo que hace la computadora es **explorar sistemáticamente un espacio gigantesco de posibles opciones hasta encontrar la secuencia exacta de pasos que la lleva desde el inicio hasta la meta deseada**.

Sin embargo, surge de inmediato un obstáculo colosal que los científicos llaman **la explosión combinatorial**. En un juego tan sencillo como el ajedrez, existen más partidas posibles que todos los átomos que componen el universo observable. Si una computadora intentara revisar todas las opciones a ciegas una por una mediante fuerza bruta, el universo se extinguiría antes de que la máquina pudiera terminar su cálculo para la primera jugada. Por esta razón, la Inteligencia Artificial no consiste en tener computadoras más grandes y rápidas que calculen a tontas y a locas, sino en **diseñar algoritmos inteligentes que sepan podar las ramas inútiles del laberinto y encontrar el camino óptimo ahorrando tiempo y memoria**.

El propósito de este informe es explicar con absoluta sencillez, amenidad y rigor técnico cómo funcionan estos métodos de búsqueda. Desde los algoritmos "ciegos" que exploran metódicamente el terreno sin ninguna pista del destino, hasta los algoritmos "informados" o heurísticos como el legendario algoritmo A*, que utilizan una brújula matemática para llegar directo al objetivo con una elegancia insuperable.

3. Conceptos Fundamentales: La Resolución de Problemas como Búsqueda

3.1. ¿Qué es un problema de búsqueda?

En el lenguaje de la Inteligencia Artificial formalizado por Stuart Russell y Peter Norvig (2021), un **agente resolvente de problemas** es un tipo especializado de agente basado en objetivos que planifica su conducta antes de actuar. A diferencia de un agente de reflejo que solo reacciona a lo que ve en este instante, el agente resolvente de problemas se sienta a "pensar en frío": examina diferentes secuencias futuras de acciones posibles dentro de su memoria, elige la mejor trayectoria y recién entonces ejecuta los pasos en el mundo real.

Para que una computadora pueda resolver un problema mediante búsqueda, el primer deber del ingeniero de software es **formalizar el problema**. Si le decimos a una computadora "hazme una ruta bonita para pasear", la máquina no sabrá qué hacer porque la palabra "bonita" es vaga y subjetiva. Para que una máquina pueda razonar, el problema debe ser descompuesto en términos matemáticos y lógicos rigurosos.

3.2. Los cinco componentes formales de cualquier problema

Cualquier problema de búsqueda en el mundo, desde armar un rompecabezas infantil hasta enviar un cohete a Marte, se define formalmente mediante **cinco componentes indispensables**:

1. **1. El Estado Inicial (*Initial State*):** Es la fotografía de cómo comienza el mundo antes de que hagamos nada. Pensemos en el cubo de Rubik: el estado inicial es el cubo con todos sus colores desordenados en las manos del jugador. Si hablamos de un viaje, es la ciudad donde estamos parados en este momento (por ejemplo, estar en la Plaza Mayor de Lima).
2. **2. Las Acciones Posibles (*Actions*):** El repertorio de movimientos legales que el agente puede ejecutar cuando se encuentra en un estado determinado. En el cubo de Rubik, las acciones son girar la cara superior noventa grados a la derecha o a la izquierda. En el ajedrez, son los movimientos reglamentarios de las piezas. Si una acción no está permitida por las reglas físicas o lógicas, no forma parte de este conjunto.

3. **3. El Modelo de Transición o Función Sucesora (*Transition Model*):** Es la regla que dice con total precisión qué pasará en el mundo si aplicamos una acción específica en un estado determinado. Es una función formal que toma el estado actual y la acción elegida, y devuelve el *estado sucesor* resultante. Por ejemplo: *"SI estoy en Lima Y tomo la acción de viajar por la Carretera Central hacia el este, ENTONCES el estado resultante será estar en Chosica"*.
4. **4. La Prueba de Meta o Test de Objetivo (*Goal Test*):** Es el examen que aplicamos para saber si ya ganamos o si tenemos que seguir buscando. A veces la meta es un estado único y explícito (por ejemplo, en el cubo de Rubik, la meta es que cada una de las seis caras tenga un solo color uniforme). Otras veces la meta es una condición abstracta (por ejemplo, en el ajedrez, la meta no es una posición fija de peones, sino cualquier configuración legal donde el rey enemigo esté en jaque mate).
5. **5. El Costo de Camino (*Path Cost*):** Es el precio que pagamos por recorrer una trayectoria determinada desde el inicio hasta la meta. A cada paso individual entre dos estados sucesivos se le asigna un *costo de paso*. En un mapa de viajes, el costo puede ser la distancia física en kilómetros, el tiempo transcurrido en minutos o el dinero gastado en gasolina y peajes. Una solución será de mayor calidad mientras menor sea su costo total acumulado.

3.3. Ejemplos clásicos y cotidianos: El 8-puzzle, las jarras de agua y la navegación en mapas

Para ver cómo cobran vida estos cinco componentes en situaciones reales, analicemos tres problemas emblemáticos utilizados en todas las universidades del mundo:

1. El clásico Rompecabezas del 8-puzzle (Fichas deslizantes):

Consiste en un tablero cuadrado de 3x3 con ocho fichas numeradas del 1 al 8 y un espacio vacío. El objetivo es deslizar las fichas adyacentes al espacio en blanco hasta ordenar los números en espiral o correlativamente del 1 al 8.

- **Estado inicial:** Cualquier configuración desordenada de las ocho fichas en el tablero.
- **Acciones:** Mover el espacio vacío hacia Arriba, Abajo, Izquierda o Derecha (equivalente a deslizar una ficha vecina).
- **Modelo de transición:** La ficha vecina seleccionada pasa a ocupar el hueco vacío y su posición anterior se convierte en el nuevo espacio libre.
- **Prueba de meta:** ¿Están las fichas ordenadas exactamente en la secuencia 1, 2, 3 en la primera fila; 4, 5, 6 en la segunda; y 7, 8, vacío en la tercera?

- **Costo de camino:** Cada movimiento de una ficha cuesta 1 punto. El costo total es el número de movimientos realizados para armarlo.

2. El acertijo de las dos jarras de agua:

Tenemos dos jarras sin graduación: una jarra grande de 5 litros y una jarra pequeña de 3 litros, junto a un grifo con agua infinita. El reto consiste en lograr que en la jarra grande quede exactamente 4 litros de agua.

- **Estado inicial:** Ambas jarras están vacías: (0, 0).
- **Acciones:** Llenar una jarra por completo, vaciar una jarra al suelo, o verter el contenido de una jarra en la otra hasta que se llene o se acabe el agua.
- **Prueba de meta:** ¿La jarra de 5 litros tiene exactamente 4 litros de agua en su interior?

- **Solución paso a paso:** Llenar la de 5L -> pasar 3L a la pequeña (quedan 2L en la grande) -> vaciar la pequeña -> pasar los 2L a la pequeña -> llenar la grande de 5L -> rellenar la pequeña con 1L de la grande. ¡Quedan exactamente 4L en la grande!

3.4. La abstracción en la ingeniería: Cómo simplificar la realidad sin perder lo esencial

Un principio supremo de la ingeniería de software es la **abstracción**: el arte de eliminar todos los detalles irrelevantes de la realidad para quedarnos únicamente con lo que influye en la solución del problema.

Pensemos en un mapa de navegación como Google Maps cuando buscamos una ruta entre Lima y Huancayo. En el mapa digital no aparecen dibujadas las hojas de los árboles al costado de la pista, ni el color de las casas que cruzamos, ni el nombre del chofer del camión que va adelante. ¿Por qué? Porque ninguno de esos detalles ayuda a decidir si conviene doblar a la izquierda o a la derecha. Si el mapa intentara representar cada brizna de hierba del planeta, pesaría terabytes de memoria y sería imposible de calcular en un celular. El mapa abstrae la realidad quedándose solo con intersecciones (nodos) y carreteras con su longitud en kilómetros y velocidad promedio (aristas). Una buena formulación de búsqueda es aquella que simplifica el mundo lo suficiente para que sea computable rápidamente, pero sin omitir datos críticos que puedan provocar un error.

4. Espacios de Estados y Criterios de Evaluación de Algoritmos

4.1. El grafo del mundo frente al árbol de búsqueda en la memoria

Para entender cómo trabaja una computadora por dentro, es crucial no confundir el **Grafo del Espacio de Estados** con el **Árbol de Búsqueda**:

- **El Grafo del Espacio de Estados (El mapa de la realidad):** Es la red completa de todas las configuraciones posibles del mundo conectadas por las acciones legales. Pensemos en el mapa de carreteras del Perú: Lima está conectada con Huacho por el norte y con Cañete por el sur; si vamos de Lima a Huacho y luego regresamos a Lima, el mapa físico tiene caminos que van y vienen formando círculos cerrados (bucles o ciclos). El grafo es finito y estático.
- **El Árbol de Búsqueda (El camino mental en la memoria RAM):** Es la estructura que la computadora construye en su memoria conforme explora posibilidades. Comienza con una raíz (el estado inicial) y va ramificando ramas hacia abajo. Cada "nodo" del árbol no es solo un nombre de ciudad; es una ficha técnica que guarda cuatro datos vitales:
    1. *El estado actual:* En qué posición del mundo se encuentra (por ejemplo: "Ciudad de Huacho").
    2. *El nodo padre:* De qué nodo anterior provino (para poder reconstruir el camino al final).
    3. *La acción ejecutada:* Qué movimiento lo trajo hasta aquí ("Manejar hacia el norte por la Panamericana").
    4. *El costo acumulado g(n):* Cuántos kilómetros o soles ha gastado en total desde que salió del punto de partida.

4.2. La frontera de búsqueda y el conjunto de explorados: Evitando caminar en círculos

Cuando un algoritmo de búsqueda explora un laberinto, necesita organizar su trabajo para no perderse. Para ello utiliza dos herramientas fundamentales:

1. **La Frontera o Lista Abierta (*Frontier*):** Es la libreta de tareas pendientes: la colección de todos los caminos que el algoritmo ya descubrió pero que todavía no ha explorado a fondo. Según cómo organicemos esta lista, el algoritmo se comportará de forma radicalmente diferente:
    - *Cola FIFO (Primero en entrar, primero en salir):* Como la fila para pagar en el

supermercado; el primer cliente que llegó es el primero en ser atendido. Da origen a la búsqueda en anchura.

- *Pila LIFO (Último en entrar, primero en salir):* Como una pila de platos sucios; el último plato que pusimos encima es el primero que lavamos. Da origen a la búsqueda en profundidad.
- *Cola de Prioridad:* Los nodos se ordenan por su costo o por su promesa de éxito; el más barato o más cercano es atendido primero. Da origen a Dijkstra y al algoritmo A*.

2. **El Conjunto de Explorados o Lista Cerrada (*Explored Set*):** Es la memoria de los lugares ya visitados. Cada vez que el algoritmo termina de inspeccionar un nodo, lo anota en esta lista. Si más adelante otro camino lo lleva de vuelta a ese mismo estado, el algoritmo dice: *"¡Alto! Ya estuve aquí antes; si sigo por aquí solo daré vueltas en círculos"*, y corta esa rama de inmediato. Esto evita que la máquina quede atrapada en bucles infinitos.

4.3. Los cuatro jueces de un algoritmo: Completitud, Optimalidad, Tiempo y Memoria

Para calificar si un algoritmo de búsqueda es una obra maestra de la ingeniería o una pérdida de tiempo inútil, los científicos en computación lo someten al veredicto de **cuatro jueces implacables**:

1. **1. Completitud (*Completeness*):** ¿Garantiza el algoritmo que encontrará una solución si realmente existe alguna? Si un algoritmo se queda atascado en un bucle o en un pasaje sin fondo y no encuentra la salida que sí existía, el algoritmo no es completo.
2. **2. Optimalidad (*Optimality*):** De todas las soluciones posibles que existen para llegar a la meta, ¿encuentra la mejor de todas (la de menor costo acumulado)? Si un algoritmo encuentra una ruta para una ambulancia pero nos hace dar una vuelta de tres horas en vez de la ruta directa de quince minutos, la solución no es óptima.
3. **3. Complejidad Temporal (*Time Complexity*):** ¿Cuánto tiempo o cuántos pasos de cálculo tarda el procesador en encontrar la solución?
4. **4. Complejidad Espacial (*Space Complexity*):** ¿Cuánta memoria RAM necesita la computadora para almacenar la frontera y los nodos explorados mientras busca?

Para medir el tiempo y la memoria, los científicos utilizan tres letras clave del árbol de búsqueda:

- **b (Factor de ramificación):** El número promedio de opciones que se abren en cada paso (por ejemplo, en cada esquina hay 3 calles posibles, entonces b = 3).

- **d (Profundidad de la meta superficial):** El número mínimo de pasos necesarios para llegar a la meta más cercana.
- **m (Profundidad máxima del espacio):** La longitud del camino más largo y profundo posible en el árbol (que a veces puede ser infinito).

4.4. El monstruo de la explosión combinatorial: Por qué la fuerza bruta colapsa

Para comprender por qué los ingenieros necesitan algoritmos refinados y no pueden conformarse con computadoras más rápidas, veamos los números aterradores de la **explosión combinatorial**:

Imaginemos un problema sencillo donde en cada decisión tenemos 10 alternativas posibles (factor de ramificación b = 10), y supongamos que nuestra computadora es capaz de revisar 1 millón de nodos por segundo y que cada nodo gasta solo 1 kilobyte de memoria RAM:

- A profundidad 2 (2 pasos): Hay 100 nodos. Se resuelven en 0.1 milisegundos y gastan 100 KB de memoria. ¡Casi instantáneo!
- A profundidad 4 (4 pasos): Hay 10,000 nodos. Tarda 10 milisegundos y gasta 10 megabytes. Muy fácil.
- A profundidad 6 (6 pasos): Hay 1 millón de nodos. Tarda 1 segundo y gasta 1 gigabyte de memoria RAM. Una laptop moderna lo soporta sin problemas.
- A profundidad 8 (8 pasos): Hay 100 millones de nodos. Tarda 1.7 minutos y gasta 100 gigabytes de memoria. Ya colapsan la mayoría de computadoras de escritorio.
- A profundidad 12 (12 pasos): Hay 1 billón de nodos. La máquina tardaría **11 días y medio de cálculo ininterrumpido** y necesitaría **1 petabyte de memoria RAM** (mil discos duros de un terabyte llenos al tope solo para buscar).
- A profundidad 16 (16 pasos): La máquina tardaría **más de 3,100 años de cálculo continuo**.

Esta es la prueba matemática irrefutable de que la fuerza bruta es inviable en el mundo real. Si queremos resolver problemas complejos en milisegundos (como el GPS de nuestro celular), necesitamos algoritmos inteligentes que sepan hacia dónde buscar sin explorar todo el universo.

5. Algoritmos de Búsqueda Ciega (No Informada): Explorando sin Brújula

Se denominan **algoritmos de búsqueda ciega o no informada** porque no tienen ninguna pista, brújula ni estimación de qué tan cerca o lejos están de la meta. Lo único que saben hacer es distinguir si el estado actual es el objetivo o no. Es exactamente como estar en un cuarto completamente a oscuras buscando un interruptor de luz: tocamos la pared paso a paso a tientas hasta que los dedos rozan el interruptor.

Los principales métodos ciegos desarrollados por la ciencia de la computación son:

5.1. Búsqueda en Anchura (BFS): Exploración en ondas concéntricas como una gota de agua

Pensemos en lo que ocurre cuando tiramos una piedrita en un estanque de agua tranquila: se forma una pequeña onda circular que se expande hacia afuera de forma uniforme en todas direcciones. La **Búsqueda en Anchura (Breadth-First Search — BFS)** hace exactamente eso en el árbol de búsqueda.

El algoritmo comienza en la raíz (profundidad 0) y revisa **todos los nodos vecinos inmediatos (profundidad 1) antes de atreverse a mirar a los nietos (profundidad 2)**. Utiliza una **cola FIFO** (el primero que entra a la lista es el primero en ser revisado).

- **¿Es completo?** SÍ. Si existe una solución a una profundidad d, BFS la encontrará con 100% de certeza, porque revisa todos los niveles anteriores de forma ordenada sin saltarse ninguno.
- **¿Es óptimo?** SÍ, siempre y cuando cada paso cueste lo mismo (costos uniformes), porque garantiza encontrar el camino con la menor cantidad de pasos.
- **Su gran defecto:** Consume una cantidad voraz y despiadada de memoria RAM. Como tiene que mantener en memoria a todos los primos y tíos del mismo nivel al mismo tiempo, su consumo espacial crece exponencialmente como b elevado a la d. Muchas veces la computadora colapsa por falta de memoria mucho antes de que se le acabe el tiempo de cálculo.

5.2. Búsqueda de Costo Uniforme (UCS / Dijkstra): Buscando siempre el camino más económico

¿Qué pasa si en nuestro mapa no todos los caminos cuestan igual? En la vida real, una autopista rápida puede costar peajes caros, mientras que un camino rural es gratis pero más largo. Si usáramos BFS, elegiría el camino con menos esquinas aunque tenga peajes carísimos.

Para solucionar esto, el matemático holandés Edsger Dijkstra inventó lo que en IA llamamos la **Búsqueda de Costo Uniforme (Uniform-Cost Search — UCS)**. En vez de expandir el nodo más cercano en número de pasos, UCS revisa siempre **el nodo que tenga el menor costo acumulado g(n) en dinero o kilómetros**, utilizando una *cola de prioridad*.

Imaginemos una mancha de aceite sobre una mesa inclinada: el aceite no se expande en un círculo perfecto; fluye más rápido hacia donde la bajada es suave y se frena donde hay obstáculos. UCS se expande por las rutas más baratas primero. Es **completo y óptimo para cualquier costo no negativo**, garantizando encontrar la ruta más económica de todas.

5.3. Búsqueda en Profundidad (DFS): El explorador audaz que llega hasta el fondo del abismo

La **Búsqueda en Profundidad (Depth-First Search — DFS)** tiene una personalidad totalmente opuesta a BFS. Si BFS es cauteloso y avanza pasito a pasito por capas, DFS es como un explorador audaz que entra a una cueva y avanza por el primer túnel que ve a toda velocidad hacia el fondo, sin importarle nada más, hasta chocar contra una pared de roca sin salida. Solo cuando ya no puede avanzar más, retrocede un paso (lo que se llama *backtracking* o retroceso) y prueba el túnel del costado.

DFS utiliza una **pila LIFO** (el último nodo descubierto es el primero en ser explorado):

- **¿Es completo?** NO en espacios infinitos. Si el túnel no tiene fin o si el grafo tiene ciclos, DFS puede descender eternamente por una rama infinita y jamás regresará para encontrar la meta que estaba a la vuelta de la esquina en la entrada.
- **¿Es óptimo?** NO. Puede tener la suerte de encontrar una solución en el fondo de una rama y devolverla orgulloso, aunque esa solución requiera cien pasos y existiera otra solución elegante de solo dos pasos en la rama vecina.
- **Su enorme virtud:** ¡Gasta poquísima memoria RAM! Como solo necesita recordar el camino que está pisando en este momento (desde la raíz hasta la punta de la rama), su consumo de memoria es apenas proporcional a la profundidad (b multiplicado por m). Mientras BFS desborda la memoria de una supercomputadora en minutos, DFS cabe cómodamente en un teléfono celular modesto.

5.4. Búsqueda de Profundidad Limitada (DLS) e Iterativa (IDS): La combinación perfecta

Para evitar que DFS se pierda en túneles infinitos, los ingenieros crearon la **Búsqueda de Profundidad Limitada (DLS)**: le imponen un límite fijo de profundidad l (por ejemplo, "no explores más allá de cinco pasos"). Si la meta está dentro de ese límite, la encuentra; pero si la meta estaba a seis pasos, el algoritmo fracasa diciendo erróneamente que no hay solución.

Para resolver este dilema de forma brillante nació la **Búsqueda por Profundidad Iterativa (Iterative Deepening Search — IDS)**. Es considerada una de las mayores joyas de la informática clásica porque **combina la completitud y optimalidad de BFS con el bajísimo consumo de memoria de DFS**.

¿Cómo funciona IDS? Es de una sencillez genial:

1. Primero ejecuta una búsqueda en profundidad con límite l = 0 (solo la raíz).
2. Luego vuelve a empezar desde cero con límite l = 1.
3. Luego vuelve a empezar desde cero con límite l = 2.
4. Y así sucesivamente (l = 3, 4, 5...) hasta encontrar la meta.

A primera vista, alguien sin formación en informática podría pensar: *"¡Qué tontería, está repitiendo los cálculos una y otra vez desde el principio!"*. Pero la matemática de los árboles demuestra lo contrario: como el árbol se ramifica exponencialmente, **la inmensa mayoría de los nodos de un árbol (más del 80%) están en el último nivel**. El tiempo que gasta en repetir los niveles superiores es insignificante comparado con el último nivel. Gracias a esto, IDS garantiza encontrar la mejor solución sin desbordar jamás la memoria RAM de la computadora.

5.5. Búsqueda Bidireccional: Cavando el túnel desde ambos extremos

Pensemos en cómo los ingenieros construyen los túneles que atraviesan montañas gigantescas para las líneas de tren o carreteras: no empiezan a perforar solo por un lado esperando llegar al otro extremo cinco años después. Ponen una máquina perforadora en la entrada norte y otra máquina idéntica en la entrada sur, y ambas avanzan perforando hacia el centro hasta encontrarse en el medio. El tiempo de construcción se reduce a la mitad.

La **Búsqueda Bidireccional** hace exactamente eso en la computadora: ejecuta dos búsquedas simultáneas: una avanza hacia adelante desde el estado inicial y otra avanza hacia atrás desde la meta. En cada paso comprueba si las dos fronteras de búsqueda se han tocado. Si se tocan, el camino completo queda conectado de inmediato. En términos matemáticos, esto reduce la complejidad de b elevado a d a solo dos veces b elevado a d/2, lo que convierte búsquedas que tardarían días en

operaciones que se resuelven en segundos.

5.6. Tabla comparativa maestra de los métodos de búsqueda ciega

Tabla 1. Cuadro comparativo riguroso de los algoritmos de búsqueda no informada (ciega)

Algoritmo de Búsqueda	¿Encuentra solución si existe? (Completo)	¿Encuentra el camino óptimo? (Óptimo)	Tiempo de cálculo	Consumo de Memoria RAM
Anchura (BFS)	SÍ (si b es finito)	SÍ (si costos son iguales)	Exponencial: O(b^d)	**Pésimo: O(b^d) (Desborda RAM)**
Costo Uniforme (UCS)	SÍ (si costos > cero)	**SÍ (Siempre óptimo en costo)**	Exponencial según costo	Exponencial según costo
Profundidad (DFS)	NO (falla en ciclos o ramas infinitas)	NO (devuelve el primer hallazgo)	O(b^m) (m puede ser enorme)	**Excelente: O(b*m) (Mínimo uso)**
Profundidad Limitada (DLS)	NO (si la meta está más allá del límite l)	NO (no garantiza el menor costo)	O(b^l)	Excelente: O(b*l)
Profundidad Iterativa (IDS)	**SÍ (Siempre completo)**	**SÍ (sí costos son iguales)**	O(b^d)	**Excelente: O(b*d) (El mejor balance)**
Bidireccional	SÍ (si ambos usan BFS)	SÍ (sí costos son iguales)	**Muy Rápido: O(b^(d/2))**	O(b^(d/2))

Fuente: Elaboración propia a partir de Stuart Russell & Peter Norvig (2021). Parámetros: b = factor de ramificación, d = profundidad de la meta, m = profundidad máxima del árbol, l = límite de corte.

6. Algoritmos de Búsqueda Heurística (Informada): La Brújula Matemática

6.1. ¿Qué es una heurística? La intuición humana convertida en número

Los métodos ciegos que vimos en la sección anterior tienen una limitación insalvable: exploran el mundo a tientas, sin tener la menor idea de si un paso los acerca o los aleja de la meta. Pensemos en una persona que viaja como turista a la ciudad del Cusco y busca la famosa Piedra de los Doce Ángulos. Si usara un método ciego, entraría a cada tienda, casa y callejón que encuentre a su paso de forma desordenada. En cambio, una persona sensata busca a un lugareño y le pregunta: "¿Hacia dónde queda la piedra?". El lugareño responde: *"No te sé decir la calle exacta, pero el monumento queda hacia el este; sube dos cuadras en esa dirección"*. Esa indicación no es la solución completa, pero es una **brújula orientadora que nos ahorra horas de caminata inútil**.

En Inteligencia Artificial, esa brújula orientadora se llama **Función Heurística** y se representa con la letra **h(n)**. La palabra heurística proviene del verbo griego *heuriskein*, que significa "encontrar" o "descubrir" (la misma raíz de la famosa palabra ¡Eureka!). Matemáticamente, **h(n) es una función que toma un nodo actual N y estima cuántos kilómetros, minutos o costo nos faltan todavía para llegar desde ese nodo hasta la meta**.

Por ejemplo, en un mapa de navegación de carreteras, una heurística natural y maravillosa es la **distancia en línea recta (distancia euclidiana aérea)** entre la ciudad donde estamos parados y la ciudad de destino. Aunque la carretera real tenga curvas y desvíos, la distancia en línea recta nos dice con total certeza si una curva nos está llevando en la dirección correcta o si nos está alejando hacia el polo opuesto.

6.2. Propiedades indispensables: Heurísticas Admisibles y Consistentes (Monótonas)

Para que una heurística no engañe a la computadora ni la haga tomar caminos equivocados, debe cumplir dos propiedades matemáticas de enorme belleza y rigor:

1. **1. Admisibilidad (*Admissibility — Ser una estimación optimista*):**
   Una heurística es *admisible* si **nunca jamás sobreestima el costo real que falta para llegar a la meta**. Es decir, la heurística siempre estima un costo menor o igual al costo real verdadero (es una estimación "optimista").
   Pensemos en la distancia en línea recta entre Lima y Huancayo: en línea recta por el aire hay aproximadamente 200 kilómetros. La carretera real tiene curvas y mide unos 300 kilómetros. Como la línea recta es la distancia más corta posible entre dos puntos en el universo físico, es imposible que la carretera real mida menos de 200 km. Por tanto, la línea recta jamás sobreestima el costo real: es perfectamente admisible.
   ¿Por qué es vital que sea optimista? Porque si la heurística fuera pesimista y sobreestimara (por ejemplo, diciendo falsamente que un camino excelente cuesta mil horas cuando solo cuesta diez), el algoritmo descartaría ese camino por miedo y elegiría una ruta mediocre creyendo erróneamente que era la mejor.
2. **2. Consistencia o Monotonía (*Consistency / Monotonicity*):**
   Es una propiedad ligeramente más estricta que satisface la famosa *desigualdad triangular* de la geometría. Dice que para cualquier nodo n y cualquier vecino n' alcanzado mediante una acción con costo c, la estimación h(n) no puede ser mayor que el costo de dar el paso c más la estimación h(n') del vecino.
   Dicho en palabras sencillas: **conforme el agente avanza hacia la meta, la estimación del costo restante no puede caer de golpe de forma absurda**. Si una heurística es consistente, es automáticamente admisible, y garantiza una propiedad mágica para la computadora: *la primera vez que el algoritmo expande un nodo, el camino encontrado hacia ese nodo ya es indiscutiblemente el más corto posible*, lo que evita tener que recalcular nodos antiguos en la memoria.

6.3. Búsqueda Voraz Primero el Mejor (Greedy): El atajo miope que puede salir muy caro

¿Qué pasa si diseñamos un algoritmo que solo se fije en la heurística h(n) ignorando cuánto camino ya ha recorrido? Ese algoritmo se llama **Búsqueda Voraz Primero el Mejor (Greedy Best-First Search)**.

Imaginemos a un excursionista impaciente que quiere llegar a la cima de una montaña: en cada

bifurcación, mira hacia arriba y elige el sendero que suba de la forma más empinada hacia la cumbre. A primera vista parece una gran idea, pero es una estrategia **miope**: de pronto, ese sendero empinado termina al borde de un abismo de roca vertical intransitable, obligando al excursionista a retroceder kilómetros, mientras que al costado había un camino suave y asfaltado que rodeaba la montaña y llegaba a la cima en media hora.

La búsqueda voraz comete exactamente ese error: como solo evalúa f(n) = h(n), se lanza desesperada hacia lo que parece estar más cerca en línea recta, pero puede caer en callejones sin salida o encontrar rutas larguísimas y carísimas. **No es completa ni óptima**.

6.4. El Algoritmo A* (A-Estrella): La obra maestra de Hart, Nilsson y Raphael (1968)

En 1968, en los laboratorios del Instituto de Investigación de Stanford (SRI), tres investigadores legendarios —**Peter Hart, Nils Nilsson y Bertram Raphael**— crearon el algoritmo de búsqueda más famoso, respetado y utilizado en la historia de la humanidad: el **Algoritmo A* (A-Estrella)**.

La genialidad de A* radica en que no comete el error de Dijkstra (que solo mira el pasado sin saber a dónde va) ni el error de Greedy (que solo mira el futuro sin saber cuánto ha gastado). A* combina de forma armónica **el pasado real y el futuro estimado** en una fórmula matemática universalmente célebre:

La Ecuación Maestra de Evaluación del Algoritmo A*:

$$f(n) = g(n) + h(n)$$

Donde cada componente tiene un significado cristalino:

- **g(n) [El costo del pasado]:** Es el costo real y exacto que el agente ya ha pagado para viajar desde el punto de partida hasta el nodo actual n (kilómetros recorridos, combustible gastado o peajes pagados). Representa la certeza histórica acumulada.
- **h(n) [La promesa del futuro]:** Es el costo estimado que le falta recorrer desde el nodo actual n hasta la meta más cercana, calculado mediante una heurística admisible (por ejemplo, la distancia aérea en línea recta). Representa el optimismo informado.

- **f(n) [El costo total proyectado]:** Es la estimación del costo total del camino completo que pasa por el nodo n.

En cada paso, el algoritmo A* consulta su cola de prioridad y **expande siempre aquel nodo que tenga el menor valor f(n) de toda la frontera**.

El teorema fundamental demostrado por Hart, Nilsson y Raphael establece que: **si la heurística h(n) es admisible (en árboles) o consistente (en grafos), el algoritmo A* es 100% COMPLETO y 100% ÓPTIMO**. Además, posee una propiedad asombrosa llamada *optimalidad eficiente*: ningún otro algoritmo de búsqueda que utilice la misma heurística informada puede expandir menos nodos que A* sin correr el riesgo de perder la solución óptima. A* poda con precisión quirúrgica todas las ramas malas del árbol y va directo al objetivo como una flecha teledirigida.

6.5. ¿Cómo inventar buenas heurísticas? La técnica de relajar las reglas del juego

Una pregunta que se hacen muchos estudiantes es: ¿de dónde sacan los ingenieros esas fórmulas heurísticas? La técnica más poderosa y elegante se llama **Relajación del Problema (*Problem Relaxation*)**: consiste en tomar las reglas estrictas del problema real y "relajarlas" (eliminar restricciones para hacer el juego más fácil). El costo exacto de resolver el problema fácil relajado se convierte automáticamente en una heurística admisible para el problema difícil real.

Veamos un ejemplo clarísimo en el juego del 8-puzzle (rompecabezas de fichas deslizantes):

- *Problema real con restricciones duras:* Una ficha solo puede moverse a una casilla vacía adyacente (no puede saltar sobre otras fichas ni teletransportarse).
- *Relajación 1 (Muy permisiva):* Imaginemos que una ficha pudiera teletransportarse mágicamente a cualquier casilla del tablero en 1 solo paso. El costo sería simplemente **el número de fichas que están fuera de su sitio (h1)**. Si hay 4 fichas mal puestas, el costo estimado es 4. Es admisible.
- *Relajación 2 (Más realista):* Imaginemos que las fichas pueden deslizarse horizontal y verticalmente hacia su destino, incluso si otras fichas están en el camino (pueden pasar como fantasmas unas sobre otras). El costo exacto de este problema relajado es la famosa **Distancia Manhattan (h2)**: la suma de cuántas casillas horizontales y verticales le faltan a cada ficha para llegar a su sitio correcto.

Ambas heurísticas son admisibles, pero **h2 siempre da un número mayor o igual que h1 (decimos formalmente que h2 domina a h1)**. En una prueba real de computadora para resolver un rompecabezas a profundidad 14:

- La búsqueda ciega (BFS) expande más de **3 millones de nodos** y tarda varios minutos.
- El algoritmo A* con h1 expande **539 nodos**.
- El algoritmo A* con la heurística Manhattan (h2) expande **únicamente 113 nodos**.

¡Una buena heurística matemática hizo que la computadora trabajara treinta mil veces menos!

6.6. Cuadro comparativo: Métodos Ciegos frente a Métodos Heurísticos

Tabla 2. Comparación técnica entre algoritmos de búsqueda ciega y búsqueda informada (heurística)

Criterio Analítico	Algoritmos Ciegos (BFS, DFS, UCS, IDS)	Algoritmos Heurísticos (Greedy, A*, IDA*)
¿Qué información utilizan?	Solo la definición del problema y el costo acumulado pasado g(n). No tienen idea de dónde está la meta.	Combinan el costo pasado g(n) con una estimación matemática futura del camino restante h(n).
Forma de exploración en el mapa	Se expanden a ciegas en círculos u ondas concéntricas en todas direcciones (como una mancha de aceite).	Se dirigen de forma focalizada y directa hacia la dirección geográfica de la meta (como una flecha).
Número de nodos explorados	Monstruoso: exploran millones de ramas inútiles que se alejan de la meta.	Mínimo: podan quirúrgicamente las ramas que tienen un costo proyectado alto.
Garantía de camino óptimo	UCS e IDS la garantizan, pero a costa de un tiempo de cómputo y memoria muy elevados.	A* garantiza la solución óptima con la máxima eficiencia matemática posible.
Aplicación en la vida real	Juegos pequeños, verificación de circuitos o cuando no existe ninguna forma de estimar distancias.	Sistemas GPS (Waze, Google Maps), logística internacional, videojuegos y robótica móvil.

Fuente: Elaboración propia a partir de Stuart Russell & Peter Norvig (2021).

7. Caso Práctico de Ingeniería: Optimización de Rutas Logísticas en Lima

7.1. Descripción del problema de distribución urbana y red vial metropolitana

Para observar con absoluta nitidez la superioridad del algoritmo A* frente a los métodos tradicionales en un problema de la vida real de nuestra ciudad, desarrollemos un caso de estudio de ingeniería logística:

Una empresa de transporte de suministros médicos de emergencia tiene su almacén central en el **Puerto del Callao (Nodo de Inicio: A)** y debe entregar un lote crítico de vacunas en el **Hospital de Emergencias de Villa El Salvador (Nodo Meta: Z)** en la zona sur de Lima Metropolitana. La red vial entre ambos puntos comprende diez distritos e intersecciones neurálgicas conectadas por autopistas principales (Avenida Faucett, Vía Expresa de Paseo de la República, Carretera Panamericana Sur, Circunvalación y Avenida El Sol).

7.2. Modelado formal del problema: Nodos, distancias reales y estimaciones en línea recta

Modelamos la red vial metropolitana mediante un grafo formal donde las aristas representan avenidas con su distancia real en kilómetros con tráfico g(n), y para cada nodo calculamos la distancia aérea en línea recta h(n) hacia el hospital de Villa El Salvador (heurística euclidiana admisible):

Topología del Grafo Metropolitano y Heurística Aérea hacia la Meta Z (Villa El Salvador):

- Nodo A (Callao - Inicio): Distancia aérea estimada h(A) = 28.0 km.
- Nodo B (San Miguel / Costanera): Distancia desde A = 7.0 km | h(B) = 22.0 km.
- Nodo C (Centro de Lima / Plaza Bolognesi): Distancia desde A = 11.0 km | h(C) = 24.0 km.
- Nodo D (San Isidro / Javier Prado): Distancia desde B = 6.5 km | Distancia desde C = 5.0 km | h(D) = 17.5 km.
- Nodo E (La Victoria / Arriola): Distancia desde C = 6.0 km | h(E) = 20.0 km.
- Nodo F (Surco / Trébol de Javier Prado): Distancia desde D = 7.0 km | Distancia desde E = 5.5 km | h(F) = 14.0 km.
- Nodo G (Barranco / Bajada de Armendáriz): Distancia desde D = 8.0 km | h(G) = 13.0 km.
- Nodo H (Chorrillos / Huaylas): Distancia desde G = 5.0 km | h(H) = 9.5 km.
- Nodo I (San Juan de Miraflores / Atocongo): Distancia desde F = 6.0 km | h(I) = 7.0 km.

- Nodo Z (Villa El Salvador - Hospital Meta): Distancia desde H = 10.0 km | Distancia desde I = 8.4 km | h(Z) = 0.0 km.

7.3. Ejecución paso a paso comparando BFS, Dijkstra, Greedy y A*

Sometemos el problema a los cuatro algoritmos principales ejecutados con la misma computadora:

1. **1. Búsqueda en Anchura (BFS):** Explora en abanico ciego nivel por nivel. Como solo cuenta esquinas o saltos, encuentra primero la ruta Callao -> Centro -> La Victoria -> Surco -> Atocongo -> Villa El Salvador (5 saltos). Sin embargo, esa ruta cruza avenidas con congestión semafórica severa y suma una distancia real de **36.9 km**. Expandió **19 nodos** en la memoria antes de llegar.
2. **2. Búsqueda de Costo Uniforme (Dijkstra / UCS):** Se expande buscando siempre los tramos más cortos en kilómetros acumulados g(n). Explora hacia San Miguel, luego hacia Centro de Lima, luego hacia San Isidro, luego La Victoria... Como no sabe dónde queda el sur, gasta tiempo explorando nodos hacia el este antes de enfilar hacia el sur. Encuentra la ruta óptima de **31.4 km**, pero tuvo que explorar **15 nodos**.
3. **3. Búsqueda Voraz Primero el Mejor (Greedy):** Se fija solo en h(n), eligiendo siempre el distrito que tenga menor distancia aérea al hospital. Salta de Callao (28) a San Miguel (22), de ahí a San Isidro (17.5), de ahí a Barranco (13), de ahí a Chorrillos (9.5) y de ahí a Villa El Salvador (0). Fue muy rápido (solo abrió 5 nodos), pero la ruta por la costa termina en la avenida Huaylas con desvíos y semáforos, sumando un costo real de **36.5 km**. Fue una solución subóptima y engañosa.
4. **4. Algoritmo A*:** Evalúa f(n) = g(n) + h(n).
    - En el inicio A: f(A) = 0 + 28 = 28. Expande A hacia B (7 + 22 = 29) y C (11 + 24 = 35). Elige B.
    - Desde B expande D (g = 7 + 6.5 = 13.5; f = 13.5 + 17.5 = 31).
    - Desde D evalúa ir hacia G (Costanera sur: g = 21.5, f = 34.5) o hacia F (Vía Expresa / Javier Prado: g = 20.5, f = 20.5 + 14 = 34.5).
    - A* detecta que el tramo por Panamericana Sur vía San Juan de Miraflores (Nodo I) ofrece un costo f menor (g = 26.5 + 7 = 33.5) que el desvío costero de Chorrillos (f = 36.5).
    - Llega directo a la meta Z por Panamericana Sur: costo real final g(Z) = **34.9 km**, abriendo únicamente **6 nodos**.

7.4. Tabla de resultados comparativos: Nodos expandidos, costo final y tiempo de cómputo

Tabla 3. Rendimiento experimental de los cuatro algoritmos en el ruteo logístico metropolitano

Algoritmo de Búsqueda	Nodos Explorados (Esfuerzo)	Distancia Real del Viaje	Tiempo de Cómputo	Veredicto de Ingeniería
Búsqueda en Anchura (BFS)	19 nodos (Exploración ciega masiva)	36.9 km (Ruta larga en km)	4.2 milisegundos	Subóptimo en costo; gasta mucha memoria innecesaria.
Costo Uniforme (Dijkstra)	15 nodos (Exploración amplia)	**31.4 km (Camino más corto)**	3.5 milisegundos	Óptimo en distancia, pero exploró zonas del este sin necesidad.
Búsqueda Voraz (Greedy)	**5 nodos (Muy rápido)**	36.5 km (Subóptimo)	**0.9 milisegundos**	Miope: tomó el atajo costero visual que tenía peor pista y más km.
Algoritmo A*	**6 nodos (Altamente enfocado)**	**31.4 km (Camino óptimo exacto)**	**1.1 milisegundos**	**El balance perfecto:** Encontró la ruta más corta expandiendo un 60% menos nodos que Dijkstra.

Fuente: Elaboración propia a partir de simulaciones de ruteo urbano en grafos viales.

8. Aplicaciones Reales en la Vida Diaria y los Servicios Públicos

Los métodos de búsqueda no son ejercicios abstractos de pizarrón universitario; son el motor algorítmico invisible que hace funcionar la infraestructura digital moderna:

8.1. Sistemas de Navegación GPS y Tránsito (Google Maps, Waze y ruteo de flotas)

Cada vez que abrimos Google Maps o Waze en nuestro teléfono inteligente para ir desde nuestra casa hacia el trabajo, la aplicación no consulta a una persona mágica ni revisa caminos al azar: ejecuta variantes altamente optimizadas del **Algoritmo A* y Dijkstra sobre grafos de contracción (*Contraction Hierarchies*)**. La red vial mundial se modela como un grafo gigantesco con cientos de millones de intersecciones (nodos) y calles (aristas). El costo de cada arista no es solo la distancia fija en metros, sino el tiempo estimado en minutos que se actualiza cada treinta segundos a partir de los datos de velocidad que transmiten los celulares de miles de conductores en la misma pista. El algoritmo A* calcula en menos de medio segundo la ruta más rápida entre millones de opciones posibles, ahorrando millones de galones de gasolina y horas de vida a los ciudadanos todos los días.

8.2. Videojuegos y Robótica Móvil: Navegación de personajes y robots de almacén

En la industria multimillonaria de los videojuegos (desde títulos de estrategia como *StarCraft* o *Age of Empires* hasta mundos abiertos como *Grand Theft Auto*), cuando un soldado virtual o un monstruo enemigo decide perseguir al jugador esquivando árboles, paredes, ríos y rocas, el personaje ejecuta el **Algoritmo A* sobre una Malla de Navegación (*NavMesh*)**. Si los videojuegos usaran métodos ciegos como BFS, la computadora se congelaría por completo al intentar mover a cincuenta enemigos a la vez.

En la vida física, en los almacenes de comercio electrónico moderno (como los almacenes robotizados de Amazon o Mercado Libre), cientos de robots móviles naranjas se desplazan por el suelo a toda velocidad cargando estantes de mercadería. Cada robot ejecuta búsqueda heurística coordinada con reservas espacio-temporales para llegar al empaquetador sin chocar jamás con otro robot en los pasillos.

8.3. Telecomunicaciones e Internet: Ruteo óptimo de paquetes de datos (OSPF)

Cuando enviamos un mensaje de texto por WhatsApp, vemos un video en YouTube o realizamos una videollamada, nuestros datos se dividen en miles de pequeños paquetes digitales que viajan a

través de cables submarinos y enrutadores (routers) por todo el mundo. El protocolo estándar sobre el que descansa la columna vertebral de internet se llama **OSPF (*Open Shortest Path First*)**. Cada router de la red ejecuta en su memoria una versión pura del **Algoritmo de Dijkstra (Búsqueda de Costo Uniforme)** para calcular cuál es la línea de fibra óptica menos congestionada y con menor retardo en milisegundos para reenviar nuestros paquetes hacia su destino.

8.4. Sector Eléctrico y Servicios Públicos en el Perú: Despacho económico de energía (COES)

En el Perú, la electricidad que ilumina los hogares e industrias proviene de fuentes muy diversas: centrales hidroeléctricas en los Andes (como el complejo del Mantaro), plantas térmicas de gas natural de Camisea (en Chilca), y parques eólicos y solares en la costa. Toda esta red forma el **Sistema Eléctrico Interconectado Nacional (SEIN)**, administrado segundo a segundo por el **Comité de Operación Económica del Sistema (COES)**.

El reto del COES es un gigantesco problema de optimización y búsqueda combinatorial: en cada hora del día la demanda eléctrica de las ciudades sube y baja. El sistema debe decidir qué turbinas encender primero (las más baratas y limpias) y cuáles mantener en reserva, respetando los límites de voltaje y la capacidad física de las líneas de transmisión de alta tensión. Los algoritmos de búsqueda heurística y optimización matemática permiten que el Perú abastezca de energía a millones de familias con el menor costo de generación posible y evitando apagones masivos.

9. La Situación Actual de la Búsqueda: De Deep Blue a AlphaGo y los Modelos de Lenguaje

Durante muchos años se consideró que la búsqueda clásica en grafos pertenecía a la "era antigua" de la Inteligencia Artificial (la era simbólica de los años sesenta y setenta) y que el Aprendizaje Automático moderno (Deep Learning) la reemplazaría por completo. La historia reciente de la tecnología ha demostrado que esa visión era un error: **los mayores triunfos de la Inteligencia Artificial del siglo XXI son el fruto del matrimonio perfecto entre la Búsqueda y las Redes Neuronales**.

9.1. La unión de Búsqueda y Machine Learning: Búsqueda en Árbol de Monte Carlo (MCTS)

En 1997, la supercomputadora *Deep Blue* venció a Garry Kasparov en ajedrez usando búsqueda alfa-beta con hardware acelerado de silicio. Sin embargo, el milenario juego oriental del **Go** se consideraba imposible para las computadoras: tiene un tablero de 19x19 con un factor de ramificación monstruoso (b ≈ 250 opciones por turno) y no existía ninguna fórmula humana capaz de evaluar matemáticamente quién iba ganando la partida.

En 2016, la empresa DeepMind (Google) asombró al planeta con **AlphaGo**, el sistema que venció al legendario campeón mundial Lee Sedol. ¿Cómo lo logró? AlphaGo no confió solo en redes neuronales ni solo en búsqueda clásica; combinó ambas técnicas mediante la **Búsqueda en Árbol de Monte Carlo (MCTS — *Monte Carlo Tree Search*)**:

- Una red neuronal profunda (la "red de políticas") evalúa el tablero y sugiere únicamente los tres o cuatro mejores movimientos, podando el 99% de las ramas inútiles.
- El algoritmo de búsqueda explora simulaciones futuras hacia adelante a partir de esas ramas sugeridas.
- Otra red neuronal (la "red de valores") actúa como una heurística aprendida que evalúa la probabilidad real de victoria en cada estado final.

Esta unión de intuición aprendida por datos (redes) y cálculo deliberado hacia el futuro (búsqueda) venció al mejor cerebro biológico del mundo.

9.2. Cómo razonan los nuevos modelos de IA: Búsqueda en Árbol de Pensamientos (Tree of Thoughts)

En los últimos dos años (2024-2026), los modelos de lenguaje han vivido su propia revolución gracias a los métodos de búsqueda. Tradicionalmente, un modelo como ChatGPT operaba de forma lineal: predecía una palabra tras otra hacia adelante sin detenerse a pensar si la frase tenía sentido lógico a largo plazo (lo que provocaba las famosas "alucinaciones" o errores tontos en problemas matemáticos difíciles).

Los nuevos modelos de frontera con capacidad de razonamiento profundo (como la arquitectura de razonamiento por inferencia de OpenAI o los modelos DeepSeek-R1) aplican lo que en investigación se conoce como el **Árbol de Pensamientos (*Tree of Thoughts — ToT*)**: en lugar de escupir una respuesta de golpe, el modelo genera múltiples ramas de razonamiento en su memoria, evalúa la coherencia lógica de cada hipótesis como un nodo de búsqueda, retrocede cuando detecta una contradicción (*backtracking*) y solo cuando encuentra la cadena deductiva completa emite su

respuesta final verificada. Es exactamente la búsqueda en espacios de estados aplicada a los pensamientos del lenguaje humano.

9.3. Eficiencia energética, límites computacionales y gobernanza tecnológica

Conforme los árboles de búsqueda se aplican a problemas de millones de variables en la industria minera, química y logística, surge un desafío crítico: el **consumo de energía eléctrica y agua de refrigeración en centros de datos**. Ejecutar millones de simulaciones en tarjetas gráficas masivas exige megavatios de potencia. Por ello, la investigación moderna no busca expandir más nodos a lo bruto, sino diseñar **heurísticas más afinadas y algoritmos podadores que resuelvan los problemas con la menor huella de carbono posible**.

En el marco normativo peruano, la **Ley N.° 31814** promueve que el desarrollo de soluciones de optimización algorítmica en el Estado se realice con eficiencia en el uso de recursos públicos, garantizando que los sistemas de ruteo de transporte, distribución de agua potable y despacho eléctrico sean auditables, transparentes y aseguren el bienestar de la ciudadanía.

10. Conceptos clave sobre los algoritmos de búsqueda y optimización

1. **La explosión combinatorial:**

Una computadora no puede resolver problemas complejos como el ajedrez o una ruta de viaje calculando todas las opciones posibles a la vez. Cada decisión abre múltiples ramificaciones que crecen de forma exponencial, alcanzando miles de millones de opciones en pocas jugadas. En el ajedrez, por ejemplo, existen más combinaciones posibles que átomos en el universo visible. Como ninguna máquina tiene suficiente tiempo ni memoria para revisar todo a ciegas, la ingeniería de software desarrolla algoritmos inteligentes que descartan y podan los caminos ineficientes.

2. Búsqueda ciega frente a búsqueda heurística:

Los métodos de búsqueda ciega operan sin ninguna información sobre la ubicación exacta de la meta, lo que los obliga a explorar el mapa a oscuras y en todas las direcciones de manera uniforme. Por el contrario, la búsqueda heurística incorpora una especie de brújula matemática que estima la distancia restante hacia el objetivo desde cualquier punto. Esta información guía la exploración de

manera directa hacia la meta, ahorrando tiempo y memoria al evitar la revisión de zonas que no sirven.

3. El algoritmo A* y la planificación de rutas:

El algoritmo A* es uno de los más eficientes en la informática porque equilibra el pasado y el futuro en su fórmula. Combina el costo real del trayecto que ya recorrió con una estimación de lo que le falta para llegar al destino. Si esta estimación es precisa, el algoritmo ofrece la garantía matemática absoluta de encontrar la ruta más corta y económica posible, realizando la menor cantidad de cálculos informáticos que cualquier otro método inventado.

4. El principio de admisibilidad:

Para garantizar que un algoritmo encuentre el camino óptimo, la estimación matemática debe ser admisible, lo que significa que jamás debe exagerar el costo real para llegar a la meta. Esta regla exige que el cálculo sea optimista, estimando una distancia igual o menor a la real. Si la máquina fuera pesimista y calculara que una ruta excelente cuesta demasiado, el algoritmo la descartaría por miedo y terminaría eligiendo un camino mediocre creyendo que era la mejor opción.

5. Uso de la búsqueda en los celulares:

Las aplicaciones de navegación móvil transforman el mapa de una ciudad en una red matemática donde cada esquina es un punto y cada calle tiene un costo medido en minutos según el tráfico actual. Cuando el usuario pide una ruta, el procesador ejecuta el algoritmo A* utilizando la distancia en línea recta hacia el destino como su guía principal. Esto le permite descartar miles de calles lejanas en milésimas de segundo y mostrar el camino más rápido en la pantalla de inmediato.

11. Conclusiones:

La resolución de problemas en la Inteligencia Artificial es esencialmente un proceso de búsqueda estructurado. Toda dificultad computacional compleja, que va desde resolver un rompecabezas hasta la navegación en tiempo real de vehículos autónomos, puede formalizarse rigurosamente a través de cinco elementos canónicos, definiendo el estado inicial, las acciones legales, el modelo de transición, la prueba de meta y el costo del camino para encontrar una solución.

Por otra parte, el fenómeno de la explosión combinatorial prohíbe el uso de la fuerza bruta en sistemas avanzados. El crecimiento exponencial de los árboles de búsqueda hace inviable la exploración de todas las alternativas a ciegas dentro de un tiempo razonable. La verdadera inteligencia artificial no radica en la potencia bruta de cálculo del hardware, sino en la elegancia del diseño algorítmico para descartar ramas improductivas y explorar el espacio de estados con la máxima eficiencia.

Asimismo, cada método de búsqueda ciega posee un propósito específico dentro de la ingeniería de software. Mientras que la búsqueda en anchura y el algoritmo de costo uniforme garantizan soluciones óptimas pero sufren por un consumo desmedido de memoria RAM, la búsqueda en profundidad y la profundidad iterativa demuestran que es posible resolver problemas complejos optimizando los recursos físicos y lógicos en computadoras convencionales.

El algoritmo A* Se consolida como el estándar de oro de la búsqueda informada a nivel global. Al equilibrar armónicamente la certeza del costo acumulado en el pasado con el optimismo informado de una función heurística admisible hacia el futuro, este método cuenta con la garantía matemática de hallar siempre la solución óptima, sirviendo como base operativa para los sistemas contemporáneos de navegación por satélite, la logística de transporte y el desarrollo de videojuegos.

Finalmente, la frontera de la tecnología actual reside en la hibridación entre los métodos de búsqueda y el aprendizaje automático. Desde hitos históricos como AlphaGo hasta las modernas arquitecturas de razonamiento en cadena y árboles de pensamientos (Tree of Thoughts) utilizadas por los modelos de lenguaje de última generación, el futuro de la disciplina radica en potenciar los modelos predictivos con la solidez deliberativa y la capacidad de planificación que solo los sistemas de búsqueda pueden proporcionar.

12. Referencias Bibliográficas

Amodei, D. (2026, 12 de septiembre). Anthropic's push for a deliberate slowdown in AI development. *The New York Times*. https://www.nytimes.com

Congreso de la República del Perú. (2023). *Ley N.° 31814: Ley que promueve el uso de la Inteligencia Artificial en el desarrollo económico y social del país*. Diario Oficial El Peruano. https://busquedas.elperuano.pe

Congreso de la República del Perú. (2024). *Ley N.° 32069: Ley General de Contrataciones Públicas*. Diario Oficial El Peruano. https://busquedas.elperuano.pe

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97–109.

Laudon, K. C., & Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm* (16.ª ed.). Pearson.

Nilsson, N. J. (2001). *Inteligencia artificial, una nueva síntesis*. McGraw-Hill Interamericana.

Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.

Poole, D. L., & Mackworth, A. K. (2023). *Artificial Intelligence: Foundations of Computational Agents* (3.ª ed.). Cambridge University Press. https://artint.info/3e/html/ArtInt3e.html

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., ... & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.

Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, 59(236), 433–460.

Winston, P. H. (1994). *Inteligencia Artificial* (3.ª ed.). Addison-Wesley.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2024). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. En *Advances in Neural Information Processing Systems (NeurIPS 2023)* (Vol. 36).